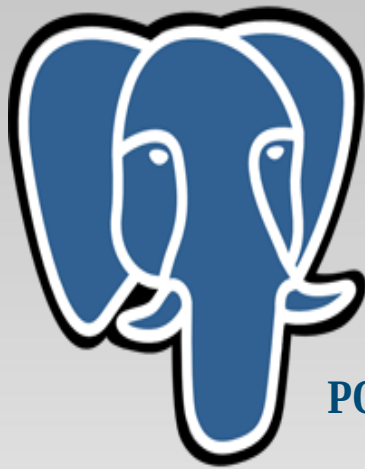




PostgreSQL

POSTGRES DATABASE ADMINISTRATION



STEPHEN NJOROGÉ

LinkedIn: www.linkedin.com/in/stephen-njoroge

| SENIOR DATABASE ADMINISTRATOR | ORACLE | POSTGRESQL | MSSQL | OCI | AWS |

CHAPTER 3: POSTGRESQL PHYSICAL STRUCTURES – DATABASES.

About the Author:

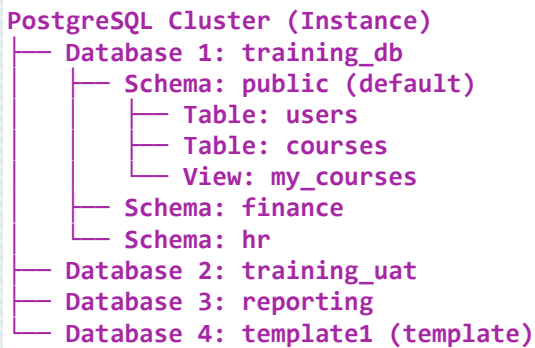
I am **Stephen Njoroge**, a Senior Database Engineer with over 8+ years of expertise in designing, securing and optimizing high-performance database systems across the banking and fintech sectors. My core technical proficiency spans major databases: Oracle, PostgreSQL, MS SQL Server and MySQL-along with advanced scripting in SQL and Python for automation and data pipeline development. I architect and manage cloud and virtualization solutions on OCI, AWS, and Proxmox for onprem systems and implement comprehensive DevOps practices using monitoring tools like Prometheus and Grafana and IT service management via iTop. Further, I deliver advanced data analytics and visualization using Tableau, Power BI and Oracle BI. As a Certified Data Protection Officer, I integrate robust data protection, compliance and security measures including vulnerability assessment and role-based access control into every layer of the data infrastructure.

Databases in PostgreSQL

Unlike some database systems (like MySQL), in PostgreSQL:

- A **database** is an **isolated container** with its own objects (tables, views, functions, etc.)
- Databases within a cluster are **logically separated** (cannot cross-query without extensions)
- Each database has its own **security permissions, encodings, and collations**

Database Hierarchy in PostgreSQL



```
postgres=# \l
```

List of databases									
Name	Owner	Encoding	Locale Provider	Collate	Ctype	Locale	ICU Rules	Access privileges	
postgres	postgres	UTF8	libc	C.UTF-8	C.UTF-8				
template0	postgres	UTF8	libc	C.UTF-8	C.UTF-8			=c/postgres +	
template1	postgres	UTF8	libc	C.UTF-8	C.UTF-8			postgres=CTc/postgres +	

(3 rows)

```
postgres=#
```

Default Databases

When you initialize a PostgreSQL cluster, you get:

```
-- List all databases
\l
```

```
postgres=# \l
```

List of databases									
Name	Owner	Encoding	Locale Provider	Collate	Ctype	Locale	ICU Rules	Access privileges	
postgres	postgres	UTF8	libc	C.UTF-8	C.UTF-8				
template0	postgres	UTF8	libc	C.UTF-8	C.UTF-8			=c/postgres +	
template1	postgres	UTF8	libc	C.UTF-8	C.UTF-8			postgres=CTc/postgres +	
(3 rows)									

```
postgres=#
```

```
SELECT datname FROM pg_database;
```

```

postgres=# SELECT datname FROM pg_database;
 datname
-----
 postgres
 template1
 template0
(3 rows)

postgres=# |

```

Creating Databases

```

-- Crating training_db database
CREATE DATABASE training_db;

```

```

postgres=# CREATE DATABASE training_db;
CREATE DATABASE
postgres=#

```

```

-- Create with owner
CREATE DATABASE training_db OWNER postgres;

```

```

postgres=# CREATE DATABASE training_db OWNER postgres;
CREATE DATABASE
postgres=#

```

```

-- Create with encoding and collation
CREATE DATABASE training_db
  ENCODING 'UTF8'
  LC_COLLATE 'C.utf8'
  LC_CTYPE 'C.utf8'
  TEMPLATE template0;

```

```

postgres=# CREATE DATABASE training_db
  ENCODING 'UTF8'
  LC_COLLATE 'C.utf8'
  LC_CTYPE 'C.utf8'
  TEMPLATE template0;
CREATE DATABASE
postgres=#

```

```

-- Create with connection limit
CREATE DATABASE training_db CONNECTION LIMIT 50;

```

```
postgres=# CREATE DATABASE training_db CONNECTION LIMIT 50;
CREATE DATABASE
postgres=# |
```

Advanced Creation Options

```
-- Create database from template
CREATE DATABASE training_db_new TEMPLATE training_db;
```

```
postgres=# CREATE DATABASE training_db_new TEMPLATE training_db;
CREATE DATABASE
postgres=# |
```

```
-- Create with tablespace
CREATE DATABASE training_db TABLESPACE training_tbs;
```

```
postgres=# CREATE DATABASE training_db TABLESPACE training_tbs;
CREATE DATABASE
postgres=#
```

```
-- Create with allow/deny connections
CREATE DATABASE training_db ALLOW_CONNECTIONS true;
```

```
postgres=# CREATE DATABASE training_db ALLOW_CONNECTIONS true;
CREATE DATABASE
postgres=# |
```

```
CREATE DATABASE maintenance_db ALLOW_CONNECTIONS false;
```

```
postgres=#
CREATE DATABASE maintenance_db ALLOW_CONNECTIONS false;
CREATE DATABASE
postgres=# |
```

Managing Databases

- List/View Databases

```
-- Basic list
\l
```

```
postgres=# \l
```

List of databases									
Name	Owner	Encoding	Locale Provider	Collate	Ctype	Locale	ICU Rules	Access privileges	
maintenance_db	postgres	UTF8	libc	C.UTF-8	C.UTF-8				
postgres	postgres	UTF8	libc	C.UTF-8	C.UTF-8				
template0	postgres	UTF8	libc	C.UTF-8	C.UTF-8			=c/postgres +	
template1	postgres	UTF8	libc	C.UTF-8	C.UTF-8			postgres=CTc/postgres +	
training_db	postgres	UTF8	libc	C.UTF-8	C.UTF-8			=c/postgres +	
training_db_new	postgres	UTF8	libc	C.UTF-8	C.UTF-8			postgres=CTc/postgres	

(6 rows)

```
postgres=#
-- Detailed list
\l+
```

List of databases											
Name	Owner	Encoding	Locale Provider	Collate	Ctype	Locale	ICU Rules	Access privileges	Size	Tablespace	
Description											
maintenance_db	postgres	UTF8	libc	C.UTF-8	C.UTF-8				7585 kB	pg_default	
postgres	postgres	UTF8	libc	C.UTF-8	C.UTF-8				7670 kB	pg_default	d
Default administrative connection database											
template0	postgres	UTF8	libc	C.UTF-8	C.UTF-8			=c/postgres +	7670 kB	pg_default	u
Nonmodifiable empty database											
template1	postgres	UTF8	libc	C.UTF-8	C.UTF-8			postgres=CTc/postgres +	7742 kB	pg_default	d
Default template for new databases											
training_db	postgres	UTF8	libc	C.UTF-8	C.UTF-8			postgres=CTc/postgres	7585 kB	pg_default	
training_db_new	postgres	UTF8	libc	C.UTF-8	C.UTF-8				7585 kB	pg_default	

(6 rows)

```
-- Using system catalog
SELECT
    datname,
    pg_size_pretty(pg_database_size(datname)) as size,
    datcollate,
    datctype,
    encoding,
    pg_encoding_to_char(encoding) as encoding_name,
    datconnlimit as conn_limit
FROM pg_database
ORDER BY pg_database_size(datname) DESC;
```

```
postgres=# SELECT
    datname,
    pg_size_pretty(pg_database_size(datname)) as size,
    datcollate,
    datctype,
    encoding,
    pg_encoding_to_char(encoding) as encoding_name,
    datconnlimit as conn_limit
FROM pg_database
ORDER BY pg_database_size(datname) DESC;
```

datname	size	datcollate	datctype	encoding	encoding_name	conn_limit
template1	7742 kB	C.UTF-8	C.UTF-8	6	UTF8	-1
postgres	7670 kB	C.UTF-8	C.UTF-8	6	UTF8	-1
template0	7670 kB	C.UTF-8	C.UTF-8	6	UTF8	-1
training_db_new	7585 kB	C.UTF-8	C.UTF-8	6	UTF8	-1
training_db	7585 kB	C.UTF-8	C.UTF-8	6	UTF8	-1
maintenance_db	7585 kB	C.UTF-8	C.UTF-8	6	UTF8	-1

(6 rows)

```
postgres=#
```

```
-- Get database sizes
SELECT
    datname as database,
    pg_size_pretty(pg_database_size(datname)) as size,
    pg_database_size(datname) as size_bytes
FROM pg_database
ORDER BY size_bytes DESC;
```

```
postgres=# SELECT
    datname as database,
    pg_size_pretty(pg_database_size(datname)) as size,
    pg_database_size(datname) as size_bytes
FROM pg_database
ORDER BY size_bytes DESC;
 database | size  | size_bytes
-----+-----+-----
 template1 | 7742 kB | 7927487
 postgres | 7670 kB | 7853759
 template0 | 7670 kB | 7853759
 training_db_new | 7585 kB | 7766543
 training_db | 7585 kB | 7766543
 maintenance_db | 7585 kB | 7766543
(6 rows)

postgres=#
```

Modify Databases

```
-- Rename database
ALTER DATABASE training_db_old RENAME TO training_db_new;
```

```
postgres=# ALTER DATABASE training_db_old RENAME TO training_db_new;
ALTER DATABASE
postgres=#
```

```
-- Change owner
ALTER DATABASE training_db OWNER TO training_user;
```

```
postgres=# ALTER DATABASE training_db OWNER TO training_user;
ALTER DATABASE
postgres=# |
```

```
-- Change connection limit
ALTER DATABASE training_db CONNECTION LIMIT 100;
```

```
postgres=# ALTER DATABASE training_db CONNECTION LIMIT 100;
ALTER DATABASE
postgres=#
```

```
-- Change default tablespace
ALTER DATABASE training_db SET TABLESPACE training_tbs;
```

```
postgres=# ALTER DATABASE training_db SET TABLESPACE training_tbs;
ALTER DATABASE
postgres=# |
```

```
SELECT
    ts.spcname as tablespace_name,
    d.datname as database_name,
    pg_size_pretty(pg_database_size(d.datname)) as db_size,
    CASE
        WHEN ts.spcname = 'pg_default' THEN 'Default tablespace'
        WHEN ts.spcname = 'pg_global' THEN 'Global tablespace'
        ELSE 'Custom tablespace'
    END as type
FROM pg_database d
JOIN pg_tablespace ts ON d.dattablespace = ts.oid
ORDER BY ts.spcname, d.datname;
```

```
postgres=# SELECT
    ts.spcname as tablespace_name,
    d.datname as database_name,
    pg_size_pretty(pg_database_size(d.datname)) as db_size,
    CASE
        WHEN ts.spcname = 'pg_default' THEN 'Default tablespace'
        WHEN ts.spcname = 'pg_global' THEN 'Global tablespace'
        ELSE 'Custom tablespace'
    END as type
FROM pg_database d
JOIN pg_tablespace ts ON d.dattablespace = ts.oid
ORDER BY ts.spcname, d.datname;
```

tablespace_name	database_name	db_size	type
pg_default	maintenance_db	7585 kB	Default tablespace
pg_default	postgres	7670 kB	Default tablespace
pg_default	template0	7670 kB	Default tablespace
pg_default	template1	7742 kB	Default tablespace
pg_default	training_db_new	7585 kB	Default tablespace
training_tbs	training_db	7585 kB	Custom tablespace

```
(6 rows)

postgres=# |
```

```
-- Change configuration parameters for specific database
ALTER DATABASE training_db SET work_mem = '16MB';
```

```
postgres=# ALTER DATABASE training_db SET work_mem = '16MB';
ALTER DATABASE
postgres=#
```

```
ALTER DATABASE training_db SET maintenance_work_mem = '256MB';
```

```
postgres=# ALTER DATABASE training_db SET maintenance_work_mem = '256MB';
ALTER DATABASE
postgres=#
```

Delete Databases

```
-- Basic drop
DROP DATABASE training_db_new;
```

```
postgres=# DROP DATABASE training_db_new;
DROP DATABASE
postgres=#
```

```
-- Drop with force (terminate connections)
```

```
ALTER DATABASE training_db ALLOW_CONNECTIONS false;
```

```
postgres=# ALTER DATABASE training_db ALLOW_CONNECTIONS false;
ALTER DATABASE
postgres=#
```

```
-- Then terminate existing connections
```

```
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE datname = 'training_db';
```

```
postgres=# ALTER DATABASE training_db ALLOW_CONNECTIONS false;
ALTER DATABASE
postgres=# SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE datname = 'training_db';
pg_terminate_backend
-----
(0 rows)

postgres=#
```

```
-- Finally drop
```

```
DROP DATABASE training_db;
```

```
postgres=# DROP DATABASE training_db;
DROP DATABASE
postgres=# |
```

Database Connections & Sessions

```
-- Who is connected to which database
```

```
SELECT
    datname,
```

```

    username,
    application_name,
    client_addr,
    state,
    query_start,
    query
FROM pg_stat_activity
WHERE datname IS NOT NULL
ORDER BY datname;

```

```

postgres=# SELECT
    datname,
    username,
    application_name,
    client_addr,
    state,
    query_start,
    query
FROM pg_stat_activity
WHERE datname IS NOT NULL
ORDER BY datname;
 datname | username | application_name | client_addr | state | query_start | query
-----+-----+-----+-----+-----+-----+-----
 postgres | postgres | psql             |             | active | 2026-01-19 15:58:21.847044+03 | SELECT
                                             | datname,
                                             | username,
                                             | application_name,
                                             | client_addr,
                                             | state,
                                             | query_start,
                                             | query
                                             | FROM pg_stat_activity
                                             | WHERE datname IS NOT NULL
                                             | ORDER BY datname;
(1 row)

postgres=#

```

```

-- Count connections per database
SELECT
    datname,
    count(*) as connections,
    count(*) FILTER (WHERE state = 'active') as active,
    count(*) FILTER (WHERE state = 'idle') as idle
FROM pg_stat_activity
WHERE datname IS NOT NULL
GROUP BY datname;

```

```

postgres=# SELECT
    datname,
    count(*) as connections,
    count(*) FILTER (WHERE state = 'active') as active,
    count(*) FILTER (WHERE state = 'idle') as idle
FROM pg_stat_activity
WHERE datname IS NOT NULL
GROUP BY datname;
 datname | connections | active | idle
-----+-----+-----+-----
 postgres |          1 |       1 |    0
(1 row)

postgres=#

```

```

-- Kill connections to a database
SELECT pg_terminate_backend(pid)

```

```
FROM pg_stat_activity
WHERE datname = 'database_to_drop';
```

```
postgres=# SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE datname = 'database_to_drop';
pg_terminate_backend
-----
(0 rows)

postgres=#
```

Database Templates.

```
-- Create database using template1 (default)
CREATE DATABASE db1;
```

```
postgres=# CREATE DATABASE db1;
CREATE DATABASE
postgres=# |
```

```
-- Create database using template0 (clean)
CREATE DATABASE db2 TEMPLATE template0;
```

```
postgres=# CREATE DATABASE db2 TEMPLATE template0;
CREATE DATABASE
postgres=#
```

```
-- Create a custom template database
CREATE DATABASE my_custom_template WITH IS_TEMPLATE = true;
```

```
postgres=# CREATE DATABASE my_custom_template WITH IS_TEMPLATE = true;
CREATE DATABASE
postgres=#
```

```
-- Now use it
CREATE DATABASE newdb TEMPLATE my_custom_template;
```

```
postgres=# CREATE DATABASE newdb TEMPLATE my_custom_template;
CREATE DATABASE
postgres=#
```

```
-- Make existing database a template
ALTER DATABASE training_db WITH IS_TEMPLATE = true;
```

```
postgres=# ALTER DATABASE training_db WITH IS_TEMPLATE = true;
ALTER DATABASE
postgres=# |
```

```
-- Prevent a database from being used as template
ALTER DATABASE training_db WITH IS_TEMPLATE = false;
```

```
postgres=# ALTER DATABASE training_db WITH IS_TEMPLATE = false;
ALTER DATABASE
postgres=#
```

Database Maintenance

Size Management

```
-- Get database size
SELECT pg_size_pretty(pg_database_size('training_db'));
```

```
training_db=# SELECT pg_size_pretty(pg_database_size('training_db'));
pg_size_pretty
-----
8902 kB
(1 row)

training_db=#
```

```
-- Get size of all objects in database
SELECT
    schemaname,
    tablename,
    pg_size_pretty(pg_total_relation_size(schemaname || '.' || tablename)) as size
FROM pg_tables
WHERE schemaname NOT IN ('pg_catalog', 'information_schema')
ORDER BY pg_total_relation_size(schemaname || '.' || tablename) DESC;
```

```
training_db=# SELECT
    schemaname,
    tablename,
    pg_size_pretty(pg_total_relation_size(schemaname || '.' || tablename)) as size
FROM pg_tables
WHERE schemaname NOT IN ('pg_catalog', 'information_schema')
ORDER BY pg_total_relation_size(schemaname || '.' || tablename) DESC;
```

schemaname	tablename	size
hr	employees	112 kB
sales	products	104 kB
finance	customers	80 kB
finance	invoices	48 kB
sales	orders	48 kB
hr	departments	40 kB
finance	transactions	32 kB
sales	order_details	32 kB
audit	change_log	32 kB
finance	accounts	32 kB
hr	job_history	16 kB

```
(11 rows)

training_db=#
```

```
-- Vacuum database
VACUUM FULL VERBOSE ANALYZE;
```

```
training_db=# VACUUM FULL VERBOSE ANALYZE;
INFO:  vacuuming "finance.transactions"
INFO:  "finance.transactions": found 0 removable, 0 nonremovable row versions in 0 pages
DETAIL:  0 dead row versions cannot be removed yet.
CPU: user: 0.00 s, system: 0.00 s, elapsed: 0.00 s.
INFO:  analyzing "finance.transactions"
INFO:  "transactions": scanned 0 of 0 pages, containing 0 live rows and 0 dead rows; 0 rows in sample, 0 estimated total rows
INFO:  finished analyzing table "training_db.finance.transactions"
avg read rate: 0.000 MB/s, avg write rate: 0.000 MB/s
buffer usage: 119 hits, 0 reads, 0 dirtied
WAL usage: 4 records, 0 full page images, 832 bytes, 0 buffers full
system usage: CPU: user: 0.00 s, system: 0.00 s, elapsed: 0.00 s
INFO:  vacuuming "finance.customers"
INFO:  "finance.customers": found 0 removable, 5 nonremovable row versions in 1 pages
DETAIL:  0 dead row versions cannot be removed yet.
CPU: user: 0.00 s, system: 0.00 s, elapsed: 0.00 s.
INFO:  analyzing "finance.customers"
INFO:  "customers": scanned 1 of 1 pages, containing 5 live rows and 0 dead rows; 5 rows in sample, 5 estimated total rows
INFO:  finished analyzing table "training_db.finance.customers"
```

```
-- Reindex entire database
```

```
REINDEX DATABASE training_db;
```

```
training_db=# REINDEX DATABASE training_db;
REINDEX
training_db=#
```

Backup/Restore Databases

```
# Backup single database
```

```
pg_dump training_db > training_db_backup.sql
```

```
postgres@998-986-02:/home/steve$ pg_dump training_db > training_db_backup.sql
postgres@998-986-02:/home/steve$ ls -lrt training_db_backup.sql
-rw-rw-r-- 1 postgres postgres 56835 Jan 19 16:23 training_db_backup.sql
postgres@998-986-02:/home/steve$
```

```
# Backup with custom format (faster restore)
```

```
pg_dump -Fc training_db > training_db_backup.dump
```

```
postgres@998-986-02:/home/steve$ ls -lrt training_db_backup.dump
-rw-rw-r-- 1 postgres postgres 140034 Jan 19 16:24 training_db_backup.dump
postgres@998-986-02:/home/steve$
```

```
# Backup all databases
```

```
pg_dumpall > all_databases.sql
```

```
postgres@998-986-02:/home/steve$ pg_dumpall > all_databases.sql
postgres@998-986-02:/home/steve$ ls -lrt all_databases.sql
-rw-rw-r-- 1 postgres postgres 66921 Jan 19 16:25 all_databases.sql
postgres@998-986-02:/home/steve$ |
```

```
# Restore single database
psql -d training_db_restored < training_db_backup.sql
```

```
postgres@998-986-02:/home/steve$ psql -d training_db_restored < training_db_backup.sql
SET
SET
SET
SET
SET
SET
set_config
-----

(1 row)

SET
SET
SET
SET
CREATE SCHEMA
ALTER SCHEMA
CREATE SCHEMA
ALTER SCHEMA
CREATE SCHEMA
ALTER SCHEMA
CREATE SCHEMA
```

```
# Restore from custom format
pg_restore -v -d db1 training_db_backup.dump 2>&1 | tee restore.log
```

```
postgres@998-986-02:/home/steve$ pg_restore -v -d db1 training_db_backup.dump 2>&1 | tee restore.log
pg_restore: connecting to database for restore
pg_restore: executing SELECT pg_catalog.set_config('search_path', '', false);
pg_restore: creating SCHEMA "audit"
pg_restore: creating SCHEMA "finance"
pg_restore: creating SCHEMA "hr"
pg_restore: creating SCHEMA "sales"
pg_restore: creating SCHEMA "staging"
pg_restore: creating TYPE "public.account_status"
pg_restore: creating DOMAIN "public.email_address"
pg_restore: creating TYPE "public.order_status"
pg_restore: creating DOMAIN "public.phone_number"
pg_restore: creating TYPE "public.transaction_status"
pg_restore: creating FUNCTION "audit.log_changes()"
pg_restore: creating FUNCTION "finance.get_customer_balance(integer)"
pg_restore: creating FUNCTION "hr.calculate_age(date)"
pg_restore: creating FUNCTION "public.update_updated_at_column()"
pg_restore: creating FUNCTION "sales.generate_monthly_sales_report(integer, integer)"
```

Database Configuration

```
-- Set parameters for specific database
ALTER DATABASE training_db SET enable_seqscan = off;
```

```
postgres=# ALTER DATABASE training_db SET enable_seqscan = off;
ALTER DATABASE
postgres=#
```

```
ALTER DATABASE training_db SET random_page_cost = 1.5;
```

```
postgres=# ALTER DATABASE training_db SET random_page_cost = 1.5;
ALTER DATABASE
```

```
-- View database-specific settings
```

```
SELECT
    datname,
    setconfig
FROM pg_db_role_setting
JOIN pg_database ON pg_database.oid = pg_db_role_setting.setdatabase;
```

```
postgres=# SELECT
    datname,
    setconfig
FROM pg_db_role_setting
JOIN pg_database ON pg_database.oid = pg_db_role_setting.setdatabase;
 datname | setconfig
-----+-----
 training_db | {enable_seqscan=off,random_page_cost=1.5}
(1 row)

postgres=# |
```

Working with Multiple Databases

```
-- Use dblink extension
CREATE EXTENSION dblink;
```

```
postgres=# CREATE EXTENSION dbLink;
CREATE EXTENSION
postgres=#
```

```
-- Query another database
```

```
SELECT * FROM dblink(
    'dbname=db1 user=postgres',
    'SELECT id, name FROM users'
) AS t1(id int, name text);
```

```

postgres=# SELECT * FROM dblink(
    'dbname=db1 user=postgres',
    'SELECT id, name FROM users'
) AS t1(id int, name text);
 id |      name
----+-----
  1 | John Doe
  2 | Jane Smith
  3 | Bob Johnson
(3 rows)

postgres=#

```

Database Security

Privileges & Permissions

```

-- Grant connect privilege
GRANT CONNECT ON DATABASE training_db TO training_user;

```

```

postgres=# GRANT CONNECT ON DATABASE training_db TO training_user;
GRANT
postgres=#

```

```

-- Revoke connect
REVOKE CONNECT ON DATABASE training_db FROM training_user;

```

```

-- Grant create privilege
GRANT CREATE ON DATABASE training_db TO training_user;

```

```

-- Grant temporary privilege (create temp tables)
GRANT TEMPORARY ON DATABASE training_db TO training_user;

```

```

postgres=# GRANT CONNECT ON DATABASE training_db TO training_user;
GRANT
postgres=# REVOKE CONNECT ON DATABASE training_db FROM training_user;
REVOKE
postgres=# GRANT CREATE ON DATABASE training_db TO training_user;
GRANT
postgres=# GRANT TEMPORARY ON DATABASE training_db TO training_user;
GRANT
postgres=#

```

```

-- View database privileges
SELECT
    datname,

```

```

    rolname,
    datacl
FROM pg_database
JOIN pg_roles ON true
WHERE datname = 'training_db';

```

```

postgres=# SELECT
    datname,
    rolname,
    datacl
FROM pg_database
JOIN pg_roles ON true
WHERE datname = 'training_db';
 datname |          rolname          | datacl
-----|-----|-----
 training_db | postgres                  | {=Tc/postgres,postgres=CTc/postgres,training_user=CT/postgres}
 training_db | pg_database_owner         | {=Tc/postgres,postgres=CTc/postgres,training_user=CT/postgres}
 training_db | pg_read_all_data          | {=Tc/postgres,postgres=CTc/postgres,training_user=CT/postgres}
 training_db | pg_write_all_data         | {=Tc/postgres,postgres=CTc/postgres,training_user=CT/postgres}
 training_db | pg_monitor                | {=Tc/postgres,postgres=CTc/postgres,training_user=CT/postgres}
 training_db | pg_read_all_settings      | {=Tc/postgres,postgres=CTc/postgres,training_user=CT/postgres}
 training_db | pg_read_all_stats         | {=Tc/postgres,postgres=CTc/postgres,training_user=CT/postgres}
 training_db | pg_stat_scan_tables       | {=Tc/postgres,postgres=CTc/postgres,training_user=CT/postgres}
 training_db | pg_read_server_files      | {=Tc/postgres,postgres=CTc/postgres,training_user=CT/postgres}
 training_db | pg_write_server_files     | {=Tc/postgres,postgres=CTc/postgres,training_user=CT/postgres}
 training_db | pg_execute_server_program | {=Tc/postgres,postgres=CTc/postgres,training_user=CT/postgres}
 training_db | pg_signal_backend         | {=Tc/postgres,postgres=CTc/postgres,training_user=CT/postgres}
 training_db | pg_checkpoint             | {=Tc/postgres,postgres=CTc/postgres,training_user=CT/postgres}
 training_db | pg_maintain               | {=Tc/postgres,postgres=CTc/postgres,training_user=CT/postgres}
 training_db | pg_use_reserved_connections | {=Tc/postgres,postgres=CTc/postgres,training_user=CT/postgres}
 training_db | pg_create_subscription    | {=Tc/postgres,postgres=CTc/postgres,training_user=CT/postgres}
 training_db | pg_signal_autovacuum_worker | {=Tc/postgres,postgres=CTc/postgres,training_user=CT/postgres}
 training_db | training_user             | {=Tc/postgres,postgres=CTc/postgres,training_user=CT/postgres}
(18 rows)

postgres=# |

```

Common Operations

-- 1. Create development and production databases

```

CREATE DATABASE myapp_dev
WITH OWNER dev_user
ENCODING 'UTF8'
CONNECTION LIMIT 20;

```

```

postgres=# CREATE DATABASE myapp_dev
WITH OWNER dev_user
ENCODING 'UTF8'
CONNECTION LIMIT 20;
CREATE DATABASE
postgres=#

```

```

CREATE DATABASE myapp_prod
WITH OWNER prod_user
ENCODING 'UTF8'
CONNECTION LIMIT 100;

```

```
postgres=# CREATE DATABASE myapp_prod
        WITH OWNER prod_user
        ENCODING 'UTF8'
        CONNECTION LIMIT 100;
CREATE DATABASE
postgres=#
```

-- 2. Create reporting read-only database

```
CREATE DATABASE myapp_reporting
        WITH OWNER reporting_user
        ENCODING 'UTF8'
        ALLOW_CONNECTIONS true;
```

```
postgres=# CREATE DATABASE myapp_reporting
        WITH OWNER reporting_user
        ENCODING 'UTF8'
        ALLOW_CONNECTIONS true;
CREATE DATABASE
postgres=# |
```

-- 3. Create template for future databases

```
CREATE DATABASE app_template
        WITH OWNER admin_user
        ENCODING 'UTF8'
        IS_TEMPLATE = true;
```

```
postgres=# CREATE DATABASE app_template
        WITH OWNER postgres
        ENCODING 'UTF8'
        IS_TEMPLATE = true;
CREATE DATABASE
postgres=# |
```

Monitoring & Health Check Queries

-- Database health dashboard

```
SELECT
    d.datname as database,
    pg_size_pretty(pg_database_size(d.datname)) as size,
    d.datconnlimit as max_conns,
    count(a.pid) as current_conns,
    count(a.pid) FILTER (WHERE a.state = 'active') as active_conns,
    max(a.query_start) as oldest_query,
    d.datfrozenxid as frozen_xid,
    age(d.datfrozenxid) as tx_age
FROM pg_database d
LEFT JOIN pg_stat_activity a ON d.datname = a.datname
WHERE d.datistemplate = false
GROUP BY d.datname, d.datconnlimit, d.datfrozenxid
ORDER BY pg_database_size(d.datname) DESC;
```

```

app_template=# SELECT
    d.datname as database,
    pg_size_pretty(pg_database_size(d.datname)) as size,
    d.datconnlimit as max_conns,
    count(a.pid) as current_conns,
    count(a.pid) FILTER (WHERE a.state = 'active') as active_conns,
    max(a.query_start) as oldest_query,
    d.datfrozenxid as frozen_xid,
    age(d.datfrozenxid) as tx_age
FROM pg_database d
LEFT JOIN pg_stat_activity a ON d.datname = a.datname
WHERE d.datistemplate = false
GROUP BY d.datname, d.datconnlimit, d.datfrozenxid
ORDER BY pg_database_size(d.datname) DESC;

```

database	size	max_conns	current_conns	active_conns	oldest_query	frozen_xid	tx_age
db1	8974 kB	-1	0	0		744	1108
training_db_restored	8894 kB	-1	0	0		744	1108
training_db	8350 kB	-1	0	0		899	953
newdb	7742 kB	-1	0	0		744	1108
postgres	7718 kB	-1	0	0		744	1108
db2	7670 kB	-1	0	0		744	1108
myapp_reporting	7585 kB	-1	0	0		744	1108
myapp_prod	7585 kB	100	0	0		744	1108
myapp_dev	7585 kB	20	0	0		744	1108
maintenance_db	7585 kB	-1	0	0		744	1108

(10 rows)

```

app_template=#

```

-- Find bloated databases (need vacuum)

```

SELECT
    datname,
    age(datfrozenxid) as tx_age,
    pg_size_pretty(pg_database_size(datname)) as size,
    CASE
        WHEN age(datfrozenxid) > 1000000000 THEN 'CRITICAL'
        WHEN age(datfrozenxid) > 500000000 THEN 'WARNING'
        ELSE 'OK'
    END as status
FROM pg_database
WHERE datistemplate = false
ORDER BY tx_age DESC;

```

```

app_template=# SELECT
    datname,
    age(datfrozenxid) as tx_age,
    pg_size_pretty(pg_database_size(datname)) as size,
    CASE
        WHEN age(datfrozenxid) > 1000000000 THEN 'CRITICAL'
        WHEN age(datfrozenxid) > 500000000 THEN 'WARNING'
        ELSE 'OK'
    END as status
FROM pg_database
WHERE datistemplate = false
ORDER BY tx_age DESC;

```

datname	tx_age	size	status
myapp_reporting	1108	7585 kB	OK
maintenance_db	1108	7585 kB	OK
db2	1108	7670 kB	OK
newdb	1108	7742 kB	OK
training_db_restored	1108	8894 kB	OK
db1	1108	8974 kB	OK
postgres	1108	7718 kB	OK
myapp_dev	1108	7585 kB	OK
myapp_prod	1108	7585 kB	OK
training_db	953	8350 kB	OK

(10 rows)

```

app_template=#

```

Database Migration/Cloning

```
# Clone database using pg_dump
pg_dump training_db | psql cloned_db
```

```
postgres=# create database cloned_db;
CREATE DATABASE
postgres=# \q
postgres@998-986-02:/tmp$ pg_dump training_db | psql cloned_db
SET
SET
SET
SET
SET
SET
SET
  set_config
-----
(1 row)

SET
SET
SET
SET
CREATE SCHEMA
```

```
# Clone database using CREATE DATABASE
psql -c "CREATE DATABASE cloned_db TEMPLATE training_db;"
```

```
postgres@998-986-02:/tmp$ psql -c "CREATE DATABASE cloned_db TEMPLATE training_db;"
CREATE DATABASE
postgres@998-986-02:/tmp$
```

```
# Clone with specific schema only
pg_dump -n public training_db | psql cloned_db
```

```
postgres@998-986-02:/tmp$ pg_dump -n public training_db | psql cloned_db
SET
SET
SET
SET
SET
SET
SET
  set_config
-----
(1 row)

SET
SET
SET
SET
ERROR:  schema "public" already exists
ALTER SCHEMA
COMMENT
```

